

During sprint 4, a static analysis tool was applied to both the front end and the back end of the web application *Meal Major* to improve code quality, enforce coding standards and detect potential issues earlier in the development. The tool that was selected for this purpose was ESLint, as it is widely used in industry for JavaScript and TypeScript, which were the main languages that the team worked with. ESLint was integrated into the codebase following standard linting rules.

After integrating ESLint into the development workflow, ESLint was executed to find existing issues. The analysis was able to identify multiple warnings and errors for both the frontend and the backend files, which included improper module structure, undefined variables, unnecessary try and catch blocks and unused variables. The issues were addressed and each one was fixed through commits.

On the frontend, ESLint detected issues with the components such as *Toast.txt* and *RecipeModal.tsx*. In both cases, the files were exporting *React* components and utility functions, which violates the principle of separation and this triggered warnings. To resolve this issue, all the non-component logic was moved into dedicated utility files, such as *toastUtils* and *recipeUtils*. This ensured that the components files only contained the logic regarding the UI. This improved modularity and aligns with ESLint.

ESLint also found issues regarding incorrect and outdated import statements in multiple files, including *ViewRecipes*, *RecipeForm*, *Profile* and *WeeklyPlanner*. After the change performed on *Toast* and *RecipeModal*, these files were still importing utility functions from the outdated component modules. The imports were updated so they import and reference the correct utility modules.

On the backend, ESLint identified more issues on the file *server*. One of the main errors was that there was a no-undef issue when a variable declared inside the try and catch block was used outside its scope. This was resolved by declaring the variable outside of the try and catch block. Another issue was also that there were no-useless-catch warnings, which meant that there was an unnecessary wrapper that provided no meaningful error handling. These were removed from the code.

Also there were multiple no-unused-var warnings for different variables. There were cases where these variables were not used intentionally, they were renamed using the convention of adding underscores. For other cases, for example where the variables were unused parameters on test files, the variables were removed or just omitted from the try and catch block. To support these fixes, the ESLint configuration was changed to include the rule regarding *varsIgnorePattern*, that allowed the variables with underscore prefixes to be ignored.

In conclusion, by using ESLint as a static analysis tool, the code was able to improve the structure and fix mistakes that have been overlooked. By addressing the modularity and runtime risks, the project had better maintainability and readability.